

Graph Query Planning Before the Query is Fully Known

Haolai Che

Ant Group, Case Western Reserve University
Shenzhen, China
hxc859@case.edu

Hanchao Ma

Case Western Reserve University
Cleveland, Ohio, USA
hxm382@case.edu

Quanqing Xu

OceanBase, Ant Group
Hangzhou, China
xuquanqing.xqq@oceanbase.com

Yinghui Wu

Case Western Reserve University
Cleveland, Ohio, USA
yxw1650@case.edu

ABSTRACT

Organizations increasingly query text-rich, schemaless graph data distributed across heterogeneous systems. Natural-language requests may leave entities, relations, or source assignments unresolved, while backend capabilities and execution costs may also be uncertain. Existing pipelines typically resolve the query first and optimize execution afterward, even though resolving an ambiguity can change the available plans and backend observations can change whether further clarification is worth its cost.

We present XGAP, a cross-platform graph query planner that plans before the query is fully known. XGAP represents unresolved query choices explicitly and jointly decides whether to acquire binding information, obtain backend metadata or statistics, construct a new physical plan, or execute. It maintains candidate interpretations together with their physical plans and updates both as observations arrive. Execution proceeds only when required validations hold and any remaining ambiguity satisfies a configurable interpretation-loss bound; physical transformations preserve the semantics of the selected query. We formulate joint information acquisition and federated planning as cost minimization under semantic, validation, and resource constraints, show that optimal policy selection is NP-hard, and develop a bounded-lookahead online algorithm with execution-correctness and interpretation-loss guarantees under stated assumptions. We implement XGAP over Neo4j and Fuseki. Experiments on social, recommendation, and financial graph workloads show that XGAP reduces end-to-end latency and execution resources while improving answer accuracy over a conventional two-stage pipeline.

PVLDB Reference Format:

Haolai Che, Quanqing Xu, Hanchao Ma, and Yinghui Wu. Graph Query Planning Before the Query is Fully Known. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

1 INTRODUCTION

Graph data rarely lives in one database. Operational deployments split identities, relationships, documents, and annotations across

property graphs, RDF stores, and application-specific services. These sources are often text-rich and lack a complete shared schema: labels and properties evolve locally, identifiers are source-specific, and backends expose different query interfaces. Natural-language requests add uncertainty about entities, predicates, and source assignments. Cross-platform planning must determine what to query while obtaining the capabilities and statistics needed to decide where and how to execute it.

Example 1: Consider a financial analyst poses a natural language query that requests “high-risk” companies related to “Alice Smith” (Figure 1). Meanwhile, the data sources that serve for high-quality answers reside separately in two sites: one uses a property graph with local query engine, and the other adopts RDF store using a SPARQL query engine. Answering the request hence non-trivially requires (1) resolving entity and relation ambiguities, and (2) validating cross-source correspondences. For example, there are two possible matches for *Alice Smith*: a_{17} (Alice J. Smith) and a_{42} (Alice R. Smith). The phrase *close financial dealings* admits two interpretations: a direct transfer of any amount, or a transfer path whose edge amounts sum to more than 100. For Alice J. Smith, account b_{38} sends 80 directly to org_{91} , satisfying the first interpretation. For Alice R. Smith, account b_{61} reaches org_{86} through b_{65} with a path total of $50 + 90 = 140 > 100$, satisfying the second. These combinations of entity binding and relation interpretation identify different companies. Company names and risk labels reside only in the RDF store. The property-graph identifiers org_{91} and org_{86} must be matched to c_7 (TechCorp) and c_{12} (FinGrp) to combine transfer matches with company names and risk labels. $\square$

Text-to-query systems resolve a request into a query, while federated optimizers choose an efficient plan for a specified query. When interpretation is optimized without execution-cost feedback, the cost of resolving ambiguity is considered separately from the cost of answering the resulting query. Yet resolving a binding changes the candidate queries and plans, and backend observations change plan feasibility and cost. These execution choices should inform whether to acquire further information, construct another plan, or execute. We therefore study joint planning of query interpretation, physical-plan construction, and execution.

We present XGAP, a cross-platform graph query planner that jointly decides what information to acquire, which physical-plan transformations to perform, and when to execute. It represents unresolved query choices explicitly in a partially specified query

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

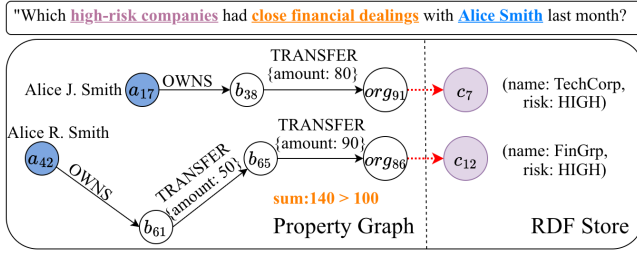


Figure 1: A cross-platform financial-risk request with two unresolved choices: which *Alice Smith*, and what counts as *close financial dealings*. Cross-source identifier mappings connect transfer matches in the property graph to company names and risk labels in the RDF store.

graph. Observations refine candidate interpretations, establish back-end capabilities, or update cost estimates. The planner uses this information to choose its next action. Execution requires all hard constraints and mandatory validation requirements to be satisfied. The selected query’s worst-case interpretation loss over the remaining candidates must not exceed the specified tolerance. Interpretation loss compares query specifications, not query results. Physical transformations preserve the semantics of the selected query. XGAP executes the plan using native query fragments and coordinator operators through existing backend interfaces, without modifying the backends’ internal optimizers.

This paper makes the following contributions.

- We formulate joint information acquisition and federated planning under validation, interpretation-loss, and resource constraints. We specify how observations change query candidates, physical-plan eligibility, and estimated costs, and prove minimum-cost policy selection NP-hard.
- We develop an online algorithm with fixed-depth lookahead [8] over information requests and physical-plan transformations. Its planning computation is polynomial under explicit representation and action limits. We establish termination, execution correctness, and interpretation-loss guarantees under the stated evidence assumptions, and a terminal-plan cost bound under bounded estimation error.
- We build XGAP, which integrates candidate validation, cost-aware clarification, and federated compilation and execution over Neo4j and Fuseki, and embeds metadata and probes into a single decision loop.
- We evaluate XGAP on workloads over real-world and synthetic graphs, varying query ambiguity and source-access requirements. Compared with a pipeline combining a text-to-query system and a federated planner, XGAP achieves lower latency and higher exact-match accuracy on requests supported by both systems.

Our source code and full version are available online [11].

Related Work. We organize the related works as follows.

Natural-language query interpretation. Natural-language interfaces address interpretation and interaction [1, 21, 30]. NaLIR exposes candidate interpretations for user correction before generating SQL [25]. MISF maintains a semantic-parsing state and decides whether and where to request user feedback [36]. Saparina and

Lapata generate alternative natural-language interpretations before semantic parsing [31]. AmbiGraph-Eval instead benchmarks LLMs on ambiguous graph queries [34]. ARUQULA generates SPARQL through iterative graph exploration and query execution over a knowledge graph [10]. These works motivate explicit treatment of ambiguity; XGAP studies the cost of obtaining information to resolve it together with downstream federated execution.

Semantic matching and query relaxation. Ontology-based subgraph querying retrieves semantically related matches using an ontology [35]. Li et al. relax failing relationship queries by retaining a maximum successful subset of the query entities [26]. These methods broaden matching or relax a specified query. XGAP instead bounds interpretation loss when selecting among candidate meanings of an unresolved request.

Federated, adaptive, and learned query optimization. BigDAWG integrates heterogeneous engines through islands of information and cross-engine data movement [17]. FedX combines source selection, join ordering, and bound joins for federated SPARQL query processing [32]. FedUP uses query-result provenance over compact federation summaries to construct result-aware query plans [3]. Robopt combines learned cost estimation with vector-based cross-platform plan enumeration [22]. Proactive re-optimization combines robust plan selection with runtime statistics collection under uncertainty [6]. Bao learns to select optimization hints for an existing optimizer [27]; Lan et al. survey cardinality estimation, cost modeling, and plan enumeration [23]. These approaches optimize the execution of a specified query. XGAP jointly plans information acquisition and federated execution under incomplete bindings and execution information, subject to an interpretation-loss bound.

Section 2 defines the data, query, and plan model. Section 3 formulates the planning problem, and Section 4 presents the algorithm and guarantees. Sections 5 and 6 describe the system and experimental study, respectively.

2 PRELIMINARIES

Source Graphs. Let $\mathcal{G} = \{G_1, \dots, G_n\}$ be a set of source graphs, with $G_i = (V_i, E_i, \ell_i, \alpha_i)$. The finite set V_i contains vertices representing entities or values. Each edge in the finite set E_i has a source and a target in V_i ; distinct edges may share endpoints. The labeling function ℓ_i assigns local labels or predicates, and the attribute map α_i assigns named values to vertices and edges. This representation accommodates property graphs and RDF graphs without requiring a shared native data model. Sources may use different identifiers and labels. We call $\mathcal{G}$ *schemaless* when it lacks a complete, enforced, and queryable global schema; local schemas and statistics may still be available. Cross-source identity or type correspondences require validated mappings or declarations.

Query Graphs and Unresolved Choices. A *partially specified query graph* Q° represents a query with unresolved choices in its graph pattern, logical source assignments, predicates, or output specification. A *complete query* Q fixes these choices. Logical source assignments identify the source graphs to query; the output specification fixes the returned expressions, grouping and aggregation, result schema, duplicates, ordering, and missing-value semantics.

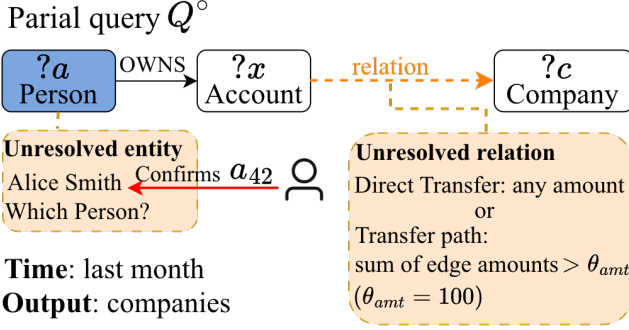


Figure 2: The partial query graph for the running request. Pattern variables describe what to match; entity and relation are unresolved choices in its constraints.

The graph pattern $G_Q = (V_Q, E_Q, f_V, f_E)$ of a complete query specifies its structure and matching constraints [14]. Here, V_Q is a finite set of pattern vertices, and E_Q is a finite set of directed edges with endpoints in V_Q . Pattern vertices denote query variables. The functions f_V and f_E assign vertex predicates and edge or path constraints, respectively. A match assigns data vertices to pattern vertices and data edges or paths to pattern edges, preserving endpoints and satisfying the constraints. Paths follow the query’s length and repetition restrictions [4]. We write $\text{Eval}(Q, \mathcal{G})$ for the result of matching the pattern over the specified source snapshots using cross-source mappings and applying the query’s predicates and output specification [15].

Let $U(Q^\circ)$ be the set of unresolved query choices. An element may specify an identity predicate in f_V , a relation or path constraint in f_E , or a choice in the other query components. These elements are parameters of the query specification, not variables ranging over data during matching. A binding β is a partial function assigning type-compatible values to these elements. It is complete when it assigns all of $U(Q^\circ)$, yielding $Q = Q^\circ[\beta]$. Completeness does not imply that the query’s bindings are validated.

Constraints and Validation. Q° carries hard constraints Γ and validation requirements, both preserved by substitution. The constraints express request requirements and application rules that every admissible complete query must satisfy. The validation requirements designate bindings, their authorized information sources, and allowed inference rules. A binding is validated either by an observation establishing it from its authorized source, or by an allowed rule that uniquely determines it from authorized observations and Γ . Model outputs, lexical matches, and ontology relations may propose or rank candidates but do not validate bindings.

Example 2: Figure 2 represents the request in Figure 1 using variables $?a$ (person), $?x$ (account), and $?c$ (company). The person owns the account; its financial relation to the company remains unspecified. Thus, $U(Q^\circ) = \{\text{entity}, \text{relation}\}$: entity identifies which Alice Smith is intended, and relation selects a direct transfer of any amount or a transfer path whose edge amounts sum to more than 100. The company must be high-risk, and each transfer must fall within last month; the query returns matching companies. User

Table 1: Logical operators used by XGAP.

Operator	Semantics
Vertex / edge match	Match vertices or single edges by predicates
Path traversal	Match paths under the query’s constraints
Selection	Retain rows satisfying a predicate
Projection	Return specified expressions
Inner equijoin	Combine rows on equal join keys
Aggregation	Compute aggregates for each group
Union	Combine compatible row sets
Ordering, limit	Order rows and restrict their number

confirmation of $\beta(\text{entity}) = a_{42}$ fixes the person identity while leaving relation unresolved. $\square$

Logical and Physical Plans. A *logical plan* is a backend-independent directed acyclic graph of operators whose edges carry rows of variable bindings and computed values. For a complete query Q , it specifies the computation of $\text{Eval}(Q, \mathcal{G})$ without choosing backend placement or data exchange. Table 1 lists the logical operators used by XGAP.

A *federated physical plan* P implements the logical computation using native backend query fragments, data-exchange operations, and coordinator operators [2]. It chooses placements for the specified logical sources and physical implementations of the operators. We write $P \equiv Q$ when P computes $\text{Eval}(Q, \mathcal{G})$ with the duplicate, ordering, and missing-value semantics specified by Q . Physical plans can be constructed for complete candidate queries before their interpretations are validated.

Example 3: Following example 2, matching identifies the person and account; traversal and selection find transfer paths within the time window whose edge amounts sum to more than 100. A join associates the endpoint companies with their risk records, and projection returns the requested companies. The sum is computed along each matched path. $\square$

Backend Descriptors. A *backend descriptor* records accessible source graphs, the native query language or API, supported operators and semantic restrictions, input and output interfaces, and available statistics. Metadata access can establish capabilities, while source probes can refine cardinality or selectivity estimates. Operator placement depends on these capabilities and the semantics required by the query [7].

Interpretation Loss. An *interpretation loss* $d(Q, \tilde{Q}) \geq 0$ measures the loss of selecting $\tilde{Q}$ when Q is intended, with $d(Q, Q) = 0$. It compares query specifications, not returned rows; a bound on this loss does not in general bound result-set error. Section 5 instantiates it with differences in query fields. The deployment specifies $\Lambda \subseteq U(Q^\circ)$, the designated elements whose bindings may remain unvalidated, and a tolerance $\epsilon \geq 0$. With $\Lambda = \emptyset$, all designated bindings require validation. Section 3 combines these requirements with physical-plan and resource checks to define execution eligibility.

Cost. Execution cost covers remaining native compilation, backend calls, data transfer, and coordinator processing through result materialization. Costs use elapsed time or fixed nonnegative resource weights. Physical optimization compares implementations of the same complete query [33]. The planning objective combines execution cost with the costs of information acquisition and selected local-planning actions (Section 3).

3 FEDERATED QUERY PLANNING UNDER INCOMPLETE INFORMATION

We formulate joint planning of information acquisition, physical-plan transformations, and execution for a partially specified query. The goal is to minimize their combined cost subject to validation, interpretation-loss, and resource constraints.

Input and State. The planner receives a partially specified query Q° , source graphs $\mathcal{G}$, and an initial state s_0 . During initialization, alternative assignments to the unresolved choices in Q° are proposed, producing complete candidate queries. Candidates violating Γ are discarded. The remaining candidates are listed in s_0 as a finite set Q_0 . Constructing these candidates does not validate their bindings; it provides alternatives whose physical plans can be considered before the intended interpretation is determined. We assume that Q_0 contains the intended query $Q^\star$, that observations used for validation are correct, and that validation follows the authorized sources and sound inference rules specified by Q° .

A planning state records the available query and execution information:

$$s = \langle \beta_s, \mathcal{E}_s, \mathcal{B}_s, \mathcal{P}_s, \mathbf{r}_s, h_s \rangle.$$

The partial binding β_s records the assignments validated so far. The evidence $\mathcal{E}_s$ consists of observations and the premises and rules used for validation. The backend descriptors $\mathcal{B}_s$ record known capabilities and statistics. For each candidate Q , $\mathcal{P}_s(Q)$ is its set of retained physical plans. The vector $\mathbf{r}_s$ records resource use since s_0 . Resource use accumulates by addition for cumulative quantities and by maxima for peak quantities. The counter h_s counts the actions performed before execution. Initially, $\mathbf{r}_{s_0} = \mathbf{0}$ and $h_{s_0} = 0$. Let $Q_s \subseteq Q_0$ contain all candidates consistent with β_s and the validation evidence in $\mathcal{E}_s$. The planner is also given the available actions, a cost model, resource budgets $\mathbf{b}^{\max}$ at s_0 , and a limit of H actions before execution.

Actions and Transitions. A nonterminal action obtains information or transforms a physical plan, after which planning continues. Let $\mathcal{A}(s)$ be the applicable nonterminal actions at s . For $a \in \mathcal{A}(s)$, $\Omega(s, a)$ is the finite set of outcomes considered in planning, and $T(s, a, o)$ is the successor state after action a produces outcome $o \in \Omega(s, a)$. An outcome is an observation returned by an information-acquisition action or a physical plan produced by a plan-transformation action.

Binding requests obtain values or validation evidence from an authorized source. Metadata requests establish backend capabilities, and source probes obtain cardinality or selectivity estimates. Metadata requests and probes may report that the requested information is unavailable. A plan-transformation action constructs an equivalent physical alternative for the same query. The terminal action $\text{Execute}(Q, P)$ executes a complete query Q using plan P

Table 2: Action Categories and Semantics

Action	Semantics
Acquire binding	validate bindings; filter candidates; drop excluded plans
Metadata	recheck capability requirements of affected plans
Probe	update a statistic; re-rank affected plans
Local plan	insert one checked plan; compute its estimate
Execute	run the selected DAG; return result and provenance

and ends planning. Table 2 summarizes the effects of these actions. Each nonterminal transition records its outcome, preserves Γ and validated bindings, and increments h_s .

Example 4: Continue Example 2 with Alice confirmed as a_{42} and the financial relation unresolved. A user reply selecting transfer paths whose edge amounts sum to more than 100 validates that interpretation and excludes the direct-transfer interpretation from Q_s . From the same state, a probe estimating the number of high-risk companies in the RDF store instead updates statistics and the costs of retained plans without changing Q_s . This probe can precede the clarification: obtaining execution information does not require first resolving the relation. $\square$

Execution Conditions. The loss function d and tolerance ϵ constrain which interpretations may execute. For $Q_s \neq \emptyset$, define the worst-case interpretation loss

$$\rho_s(\tilde{Q}) = \max_{Q \in Q_s} d(Q, \tilde{Q}).$$

The maximum ranges over all remaining candidates, regardless of their costs or whether they have retained plans. Since validation uses correct observations and sound rules, it preserves $Q^\star \in Q_s$. Consequently, $d(Q^\star, \tilde{Q}) \leq \rho_s(\tilde{Q})$.

A query-plan pair (Q, P) belongs to the eligible set $\mathcal{F}_\epsilon(s)$ if and only if all the following conditions hold: $Q \in Q_s$, $P \in \mathcal{P}_s(Q)$, validation of every designated binding outside Λ , and $\rho_s(Q) \leq \epsilon$. The required backend capabilities, interfaces, mappings, and source versions must be known to support P , and a sound compilation or rewrite rule must establish $P \equiv Q$. The plan’s remaining resource use must have valid upper bounds within the available budgets; these bounds must not underestimate consumption. If $Q_s = \emptyset$, no pair is eligible. For an eligible pair, $\text{Execute}(Q, P)$ returns $\text{Eval}(Q, \mathcal{G})$, also records $\rho_s(Q)$, and the supporting evidence.

Policy and Objective. A policy π specifies the next action at each reached state. One run of the policy produces a trace

$$\tau = (s_0, a_0, o_0, \dots, s_L, a_L, z),$$

where $L \leq H$, $s_{t+1} = T(s_t, a_t, o_t)$ for $0 \leq t < L$, and the terminal action a_L returns output z . The trace cost $C(\tau)$ is the sum of the costs of information acquisition, performed plan transformations, and execution through result materialization. Peak resources are constrained by budgets rather than added to the cost. The state contains the information needed to determine subsequent costs. Initialization and the computation used to choose actions are accounted for separately and are not part of $C(\tau)$.

Let Π_ϵ be the policies that reach eligible execution within the action and resource limits for every sequence of modeled outcomes. If an outcome distribution $p(\cdot \mid s, a)$ is supplied for every reachable state-action pair, the objective is to minimize the expected policy cost $J(\pi)$:

$$\min_{\pi \in \Pi_\epsilon} J(\pi), \quad J(\pi) = \mathbb{E}_{\tau \sim \pi} [C(\tau)]. \quad (1)$$

When outcome distributions are unavailable, we minimize the worst-case policy cost $J_{\max}(\pi) = \max_{\tau \in \text{Traces}(\pi)} C(\tau)$ over $\pi \in \Pi_\epsilon$. Here, $\text{Traces}(\pi)$ is the set of complete traces generated by π under the modeled outcomes.

Theorem 1: *Deciding whether a policy in Π_ϵ has cost at most W is NP-hard, even with one candidate query, one backend, deterministic acquisition outcomes, and one zero-cost physical plan.* $\square$

Proof sketch: Reduce from weighted set cover. Create one complete candidate query with one binding requiring validation for each universe element, and one zero-cost physical plan. Each binding has a unique candidate value. For each set, provide one action that validates exactly its elements at the set's weight. Set $\Lambda = \emptyset$, permit no other validation actions or inference rules, allow each action once, and set H to the number of sets. Other budgets are nonbinding and all other costs are zero. A cover yields a feasible policy by performing its actions and executing the plan. Conversely, a feasible policy's deterministic trace must validate every element, so its actions form a cover of the same cost. The construction of the reduction is polynomial. $\square$

4 COST-BASED FEDERATED PLANNING

XGAP interleaves information acquisition with physical-plan construction. It evaluates each nonterminal action by its estimated cost and the estimated cost of subsequent actions and execution. It compares this total with the cost of immediate execution for an eligible query-plan pair. After performing a nonterminal action, XGAP updates the planning state and chooses the next action.

Action Generation and Plan Retention. For each unresolved query choice, XGAP generates at most one action to request its binding from an authorized source. Additional information-acquisition actions come from an explicit list. They request validation of several bindings together, retrieve capability metadata, or probe a specified source fragment or predicate.

A plan-transformation action specifies a retained plan, a rule, its application position, and its parameters. The fixed rules support selection, projection, and aggregation pushdown; placement at an equivalent replica; changes to inner-join implementation; and adjacent inner-join reordering. Placements come from an explicit list of equivalent replicas. Join reordering uses commutativity or associativity [19]. A rule applies only when its conditions establish $P \equiv Q$ for the resulting plan.

XGAP retains at most $K \geq 1$ plans per candidate query, each of representation size at most ℓ . Initial plans obey these limits; transformations that produce larger plans are excluded. After a transformation, XGAP inserts the new plan into $\mathcal{P}_s(Q)$ and removes duplicate encodings. If an eligible plan exists in this pool, it keeps the one with the lowest estimated execution cost and fills up to $K - 1$ remaining positions with the cheapest other plans. This

reservation is needed because plans with lower estimates may not yet be eligible. If no plan is eligible, XGAP keeps up to K plans with the lowest estimates. A generated action enters $\mathcal{A}(s)$ only if valid upper bounds on its resource use keep cumulative and peak consumption within their budgets. These bounds cover every outcome permitted by the action.

Action Evaluation. For each retained pair (Q, P) with $Q \in \mathcal{Q}_s$, let $\widehat{c}_{\text{term}}(s, Q, P)$ be a finite nonnegative estimate of the cost of executing P for Q from state s . The least expensive eligible execution has estimated cost

$$\widehat{C}_{\text{term}}(s) = \min_{(Q, P) \in \mathcal{F}_\epsilon(s)} \widehat{c}_{\text{term}}(s, Q, P),$$

with $\min \emptyset = +\infty$. Let $\widehat{c}(s, a, o)$ be a finite nonnegative estimate of the cost of performing action a at s with outcome o .

To evaluate an action, XGAP considers each outcome $o \in \Omega(s, a)$ and the choices available at its successor $T(s, a, o)$. These decisions form an AND/OR structure [9]. A state is an OR node that selects a nonterminal action or eligible execution. A nonterminal action is an AND node with one successor for each modeled outcome. A plan transformation has a single successor.

To limit planning computation, XGAP uses lookahead of fixed depth $D \geq 1$. Each branch contains at most D nonterminal actions. Let $\widehat{V}_j(s)$ denote the estimated remaining cost computed with j levels of lookahead. For a hypothetical probe outcome, the category's representative estimate updates the corresponding statistic and affected plan costs.

Expected-cost planning uses an input probability model that supplies $p(\cdot \mid s, a)$ for every reachable state-action pair. For $j > 0$ and $h_s < H$, the recurrence compares immediate execution with the cost of each action and its continuation:

$$\widehat{V}_j(s) = \min \left\{ \widehat{C}_{\text{term}}(s), \min_{a \in \mathcal{A}(s)} \mathbb{E}_o \left[\widehat{c}(s, a, o) + \widehat{V}_{j-1}(T(s, a, o)) \right] \right\}. \quad (2)$$

Here, $\mathbb{E}_o$ denotes expectation over $o \in \Omega(s, a)$ with probabilities $p(o \mid s, a)$. Without this probability model, XGAP uses the worst-case objective throughout, replacing the expectation with the maximum over $\Omega(s, a)$.

States with contradictory validation evidence, violated hard constraints or budgets, or $\mathcal{Q}_s = \emptyset$ have $\widehat{V}_j(s) = +\infty$. An action has infinite estimated cost if any modeled successor has infinite value, regardless of its probability. These rules take precedence over the recurrence and its boundary values.

The action limit and the lookahead depth limit have different roles. At a valid state with $h_s = H$, only execution remains, so $\widehat{V}_j(s) = \widehat{C}_{\text{term}}(s)$. At the depth limit, further actions may still be needed before execution. We estimate the remaining cost from the retained plans to avoid expanding more actions. Define $g(s)$ as the minimum $\widehat{c}_{\text{term}}(s, Q, P)$ over all retained pairs with $Q \in \mathcal{Q}_s$, including pairs that are not yet eligible. Set $g(s) = 0$ if none is retained. This approximation omits the costs of further information acquisition and plan transformations. At a valid state with $j = 0$ and $h_s < H$, set $\widehat{V}_0(s) = g(s)$.

Example 5: Continue Example 4 with the path interpretation confirmed, giving the complete query Q . The only retained plans are P_{bind} and P_{hash} . P_{bind} retrieves high-risk company keys from the

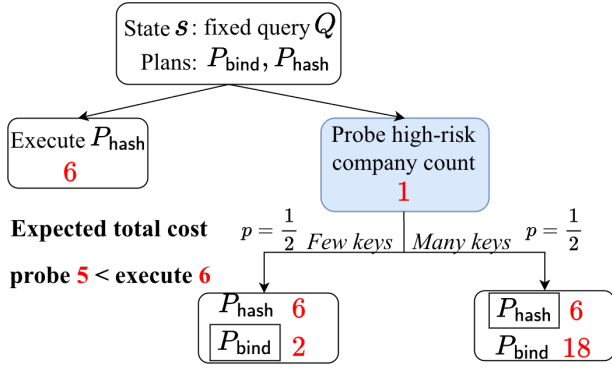


Figure 3: A probe changes which physical plan has the lowest estimated cost. XGAP compares probing followed by plan selection with immediate execution. All numbers are illustrative estimates in the same cost unit.

RDF store, maps them to property-graph identifiers, and restricts path endpoints to those identifiers. P_{hash} retrieves property-graph path matches and RDF high-risk company records independently, then performs a hash join on the mapped company keys at the coordinator. The available choices are immediate execution or probing the number of high-risk company keys in the RDF store (Figure 3). Suppose the remaining action and resource limits permit the probe followed by either plan. Both plans are eligible before and after either probe outcome. The count affects the cost of passing keys to the property graph and retrieving matching paths.

Initially, P_{bind} and P_{hash} have estimated costs 10 and 6, so $\hat{C}_{\text{term}}(s) = 6$. The probe costs 1. Its model assigns probability $1/2$ to each of two categories: few keys and many keys. For few keys, the estimates are 2 for P_{bind} and 6 for P_{hash} . For many keys, they are 18 and 6, respectively. At $D = 1$, lookahead stops at the probe’s successor states. Since both retained plans are eligible there, $g = \hat{C}_{\text{term}}$. The cheapest plan is P_{bind} for few keys and P_{hash} for many keys. The estimated total cost of probing and subsequent execution is $1 + \frac{1}{2} \cdot 2 + \frac{1}{2} \cdot 6 = 5 < 6$, so XGAP selects the probe. If the probe costs 3, the total becomes 7 and XGAP instead executes P_{hash} . $\square$

Online Planning and Execution. Algorithm 1 applies Equation 2 and its boundary rules at each round. After performing the selected nonterminal action, XGAP uses its returned outcome to update $s \leftarrow T(s, a_{\text{best}}, o)$ and replans. For a probe, this update uses the returned estimate in place of the category representative used during lookahead. It recomputes the estimated costs of affected plans. An unavailable estimate leaves the statistics and plan costs unchanged. When execution is selected or $h_s = H$, XGAP executes the eligible pair with the lowest estimated cost in the current state. Equal estimates favor execution; other ties follow a fixed order.

If every choice has infinite estimated cost or the final eligible set is empty, the request terminates without execution. It also terminates without execution if the final eligible set is empty, including when $h_s = H$. Calls enforce the admitted resource limits and specified source versions; a failed call terminates the request.

Algorithm 1 Online planning and execution in XGAP

Input: Partially specified query Q° ; $\mathcal{G}$; initial state s_0

Output: Selected (Q, P)

```

1:  $s \leftarrow s_0$ 
2: while  $h_s < H$  do
3:    $\mathcal{A}(s) \leftarrow \text{GENERATEACTIONS}(s)$ 
4:    $j \leftarrow \min\{D, H - h_s\}$ 
5:    $(a_{\text{best}}, v_{\text{best}}) \leftarrow (\text{Execute}, \hat{C}_{\text{term}}(s))$ 
6:   for all  $a \in \mathcal{A}(s)$  in fixed order do
7:      $v \leftarrow \mathbb{E}_o[\hat{c}(s, a, o) + \hat{V}_{j-1}(T(s, a, o))]$ 
8:     if  $v < v_{\text{best}}$  then
9:        $(a_{\text{best}}, v_{\text{best}}) \leftarrow (a, v)$ 
10:  if  $a_{\text{best}} = \text{Execute}$  then
11:    break
12:   $o \leftarrow \text{PERFORM}(a_{\text{best}})$ 
13:   $s \leftarrow T(s, a_{\text{best}}, o)$ 
14:  $(Q, P) \leftarrow \arg \min_{(Q, P) \in \mathcal{F}_\epsilon(s)} \hat{C}_{\text{term}}(s, Q, P)$ 
15:  $\text{EXECUTE}(Q, P)$ 
16: return  $(Q, P)$ 

```

Figure 4: Online action selection in XGAP. Each round compares eligible execution with information acquisition and plan transformations. The selected nonterminal action updates the state for the next round.

Analysis. We use the evidence assumptions of Section 3, sound plan-transformation rules, and fixed source snapshots.

Theorem 2: Every successful execution in Algorithm 1 satisfies Γ , preserves validated bindings, and meets the required validations and resource budgets. It produces $\text{Eval}(Q, \mathcal{G})$ for the selected query Q , with $d(Q^*, Q) \leq \epsilon$. $\square$

Proof sketch: Correct observations and sound validation rules preserve $Q^* \in Q_s$ by induction on transitions. Plan retention leaves Q_s unchanged. The final pair belongs to $\mathcal{F}_\epsilon(s)$, so it satisfies Γ and the binding requirements, and $P \equiv Q$. Since $Q^* \in Q_s$, the loss check gives $d(Q^*, Q) \leq \rho_s(Q) \leq \epsilon$. Resource upper bounds for admitted actions and eligible execution preserve the budgets. $\square$

Cost estimation error. Let $c_{\text{term}}(s, Q, P)$ be the true remaining execution cost. The lemma bounds the extra execution cost of selecting a plan by its estimate, for a fixed query and eligible plan set.

Lemma 3: Fix s, Q , and a nonempty eligible plan set $\mathcal{P}$. If every $P \in \mathcal{P}$ satisfies $|\hat{c}_{\text{term}}(s, Q, P) - c_{\text{term}}(s, Q, P)| \leq \eta$, then the estimate-minimizing plan has true remaining cost at most 2η above the true minimum over $\mathcal{P}$. $\square$

Proof sketch: Let $\hat{P}$ be the selected plan and P_{opt} the plan with minimum true cost in this set. The estimate for $\hat{P}$ is no larger than that for P_{opt} . The true cost of $\hat{P}$ is at most η above its estimate, and the estimate for P_{opt} is at most η above its true cost. Combining the two errors gives $c_{\text{term}}(s, Q, \hat{P}) - c_{\text{term}}(s, Q, P_{\text{opt}}) \leq 2\eta$. $\square$

Termination and complexity. Each nonterminal transition increments h_s , so XGAP performs at most H nonterminal actions before execution or request termination. Each recursive evaluation decreases j . Thus the procedure terminates if external calls return or fail in finite time. Fixed-depth lookahead may fail to find a feasible policy even when one exists.

For the complexity analysis, the input includes the listed candidate queries, action descriptions, outcome lists $\Omega(s, a)$, and allowed placements. We assume polynomial bounds on H, K, ℓ , observation record sizes, and numeric bit lengths in this input size. Response checks, loss evaluation, validation inference, resource checks, cost estimation, outcome-probability evaluation, and transformation-rule checks take polynomial time. The fixed rules and explicit request and placement lists give polynomial bounds on the numbers of actions and modeled outcomes per state.

Let A bound the number of actions per state and Δ the number of modeled outcomes per action, for both actual and hypothetical states. Let T_{local} bound the time for action generation at one state, processing one outcome, and scanning retained plans for eligibility, costs, and $g(s)$. A depth- D lookahead tree has at most $\sum_{j=0}^D (A\Delta)^j$ state nodes. With at most H rounds and one final scan for eligible execution, the total planning time is

$$O\left((H+1) \sum_{j=0}^D (A\Delta)^j T_{\text{local}}\right). \quad (3)$$

For fixed D , this bound is polynomial under the assumptions above. It covers coordinator planning, excluding backend execution and time waiting for responses.

Full proofs and detailed analysis appear in Appendix A [11].

5 XGAP SYSTEM

XGAP translates a natural-language request into candidate queries, jointly plans information acquisition and physical execution, and executes the selected query across sources (Figure 5). The joint planner implements Algorithm 1. It obtains observations through an information-acquisition module and constructs physical alternatives through a federated compiler. An execution coordinator runs the selected plan through backend adapters. The implementation uses Cypher [16] for Neo4j [28] and SPARQL for Fuseki [5].

Query Interpretation and Initialization. The query interpretation module supplies an LLM with the request and available context, such as source metadata, user specifications, or ontology relations. The LLM proposes the partially specified query graph Q° and alternative bindings for its unresolved choices. Candidate construction substitutes these bindings into Q° , checks types and hard constraints, and removes duplicate queries to form Q_0 .

The user or another authorized source confirms that the intended query is covered by this candidate set, without making a selection. The compiler constructs initial physical plans using the available backend descriptors. The candidates, plans, descriptors, and initial evidence initialize s_0 . Appendix B details candidate construction and the field-based loss function [11].

XGAP instantiates interpretation loss as the weighted fraction of query fields whose bindings differ. Each choice $e \in U(Q^\circ)$ has a

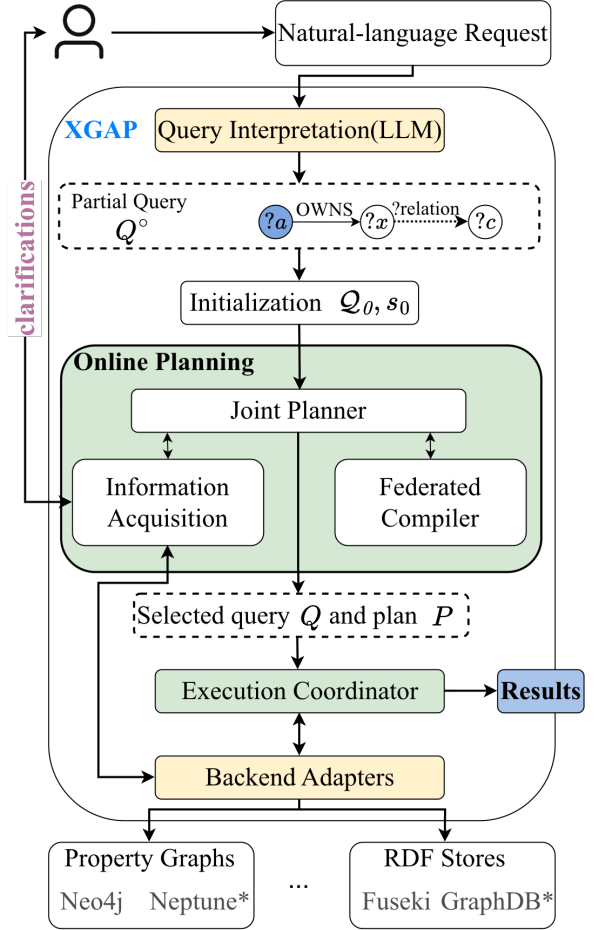


Figure 5: XGAP system architecture. The joint planner coordinates information acquisition and physical-plan construction before cross-source execution. Starred systems illustrate adapter extensions.

fixed positive integer weight w_e . For two candidate queries,

$$d(Q, Q') = \frac{\sum_{e \in U(Q^\circ)} w_e 1[Q(e) \neq Q'(e)]}{\sum_{e \in U(Q^\circ)} w_e}.$$

Here, $Q(e)$ is the value assigned by Q to choice e . The denominator stays fixed as observations resolve choices. The loss is zero when $U(Q^\circ)$ is empty.

Information Acquisition. The information-acquisition module sends binding requests to authorized sources and uses their replies to refine the candidate family. Backend requests go through the corresponding adapters. A capability request asks whether a backend supports an operator with the required input and output semantics. Its response updates plan eligibility. A probe requests a cardinality or selectivity estimate for a specified source fragment or predicate. The returned estimate updates plan costs. The cost model supplies action and execution cost estimates to the joint planner; Appendix C gives the calculations [11].

Federated Compilation. The federated compiler generates and transforms physical plans for complete candidate queries. It converts each candidate to a logical DAG of the operators in Table 1. It then partitions the DAG into source-local fragments and coordinator operators and compiles the fragments into native queries. Native fragments return rows of variable bindings and computed values through a common typed interface. Cross-source joins use canonical keys obtained from validated identifier mappings. The adapters translate between these keys and source identifiers when forming native requests and decoding responses.

The compiler pushes an operator into a native fragment only when the backend preserves its types, duplicate handling, missing values, and grouping semantics. Other operators remain at the coordinator. Each physical alternative records its compilation or transformation rule and the conditions establishing $P \equiv Q$. Replica placement requires a declaration of the same logical source and snapshot. Backend-specific code is confined to adapters and native-fragment compilation. The joint planner uses common query and physical-plan representations across backends.

Cost Estimation. The cost model uses plan structure and backend statistics to estimate execution costs [24]. A topological pass estimates intermediate cardinalities, row widths, and distinct join-key counts. Row counts and widths determine transfer estimates; operator implementations determine processing work. The estimate combines source calls and processing, data exchange, coordinator processing, and remaining compilation in planning work units.

For hash joins, costing includes reading both inputs and joining them at the coordinator. For bind joins, the number of distinct driver keys and batch capacity determine the target-call estimate. Target access paths determine the work of each batch. A returned probe estimate updates its target statistic and triggers recomputation of dependent estimates. Information-acquisition costs account for calls and disclosed fields as applicable; transformation costs cover variant construction and new compilation. Appendix C provides the formulas and units [11].

Cross-Source Execution. The execution coordinator schedules the selected DAG in dependency order. It dispatches native fragments through backend adapters and runs coordinator operators on the returned binding rows. Independent source calls run concurrently. For a hash join [18], the coordinator retrieves both inputs and joins their rows on canonical keys.

A bind join first evaluates one input, called the driver, and extracts its distinct join keys. These keys are sent to the target source in batches. Each batch restricts the target fragment using SPARQL VALUES or Cypher parameters. The coordinator combines the returned rows and joins them with the original driver. Key deduplication reduces repeated lookups; using the original driver preserves row multiplicities in the join result. Appendix B.4 in the full version [11] gives the conditions under which this restriction preserves the result. After completing the DAG, the coordinator materializes the answer. It records the selected query and plan together with the observations and resource use from the request.

6 EXPERIMENTAL STUDIES

6.1 Experimental Setup

We evaluate whether joint planning reduces the time and resource cost of answering partially specified queries while preserving answer quality. The experiments compare XGAP with an external text-to-query pipeline, isolate the effects of probing and lookahead, and examine how ambiguity, acquisition prices, and cross-source execution affect the benefits.

Platform and Language Model. Experiments run on an Intel Xeon Gold 6230 (16 cores, 64 GB RAM) under Red Hat Enterprise Linux 8.10. The coordinator runs on the same compute node as Neo4j Community 5.26.30 and Apache Jena Fuseki 5.6.0, so coordinator-backend calls are local. Neo4j executes Cypher queries [28], and Fuseki serves RDF through SPARQL endpoints [5]. The XGAP front end and the external baseline both call a Qwen 27B model (API alias qwen3.8-27b, temperature 0.0, maximum output 4096 tokens, thinking disabled, 60 s timeout) through a remote API.

Datasets. We use three public datasets from the social, recommendation, and financial domains. For each dataset, we construct graph and attribute partitions from the same source snapshot. The partitions share identifiers derived deterministically from the original identifiers. The sizes below refer to the selected source projections.

D1 (SNB-derived). The LDBC Social Network Benchmark (SNB) [13] provides synthetic social graphs. We use an SF 0.1 snapshot in CsvBasic format with LongDateFormatter and select its Person/KNOWS projection. The graph partition stores Person vertices and KNOWS relationships; the RDF attribute partition stores gender. The projection contains 1,528 vertices and 14,073 relationships.

D2 (MovieLens-derived). We use the MovieLens 20M release [20]. The graph partition stores users, movies, and rating relationships with rating values and timestamps. The RDF attribute partition stores isDrama, derived from the original movie genres. The projection contains 165,771 vertices and 20,000,263 rating relationships.

D3 (FinBench-derived). The LDBC FinBench [29] provides synthetic financial graphs. We use the account and transfer projection of the FinBench v0.1.0 snapshot at SF 0.1. The graph partition stores accounts and transfer relationships with amounts and timestamps; the RDF attribute partition stores the original account attribute isBlocked. The projection contains 20,409 accounts and 79,909 transfer relationships.

Workloads. We reuse graph patterns, predicates, aggregations, and output specifications from LC-QuAD 2.0 [12] and FinBench. Instantiating them with each dataset’s entities and values gives complete intended queries. We then hide selected fields to create partially specified requests. Each dataset has 200 query-intent cases in each of four classes: W1 (unambiguous, single source), W2 (entity ambiguity, single source), W3 (entity and predicate ambiguity, cross source), and W4 (multiple unresolved fields, including logical source scope, cross source). A simulated user holds the intended query, hidden from the LLM and planner, and answers only requested fields. An offline SQLite evaluator computes reference answers on the same source snapshot. It implements pattern matching, bounded

paths, filtering, and aggregation without using XGAP’s planner, compiler, or execution operators.

Compared Methods. We compare against a two-stage baseline (TS) that combines ARUQULA for text-to-SPARQL generation [10] with FedX 5.1.2 for federated planning and execution [32]. We run it as an external system with the shared LLM interface. Since TS accepts RDF, we provide an RDF mapping of the source data; XGAP uses the property-graph and RDF partitions. The two systems therefore query the same underlying facts through different physical representations. We retain TS’s query-generation and execution procedures. Paired comparisons use requests supported by both systems after mapping.

We also compare three XGAP variants. *XGAP-noProbe* (NP) removes statistics probes but retains capability requests. *XGAP-shallow* (SH) sets $D = 1$ and retains all action types. *XGAP-greedy* (GR) removes continuation-cost estimates from action evaluation. It compares each nonterminal action’s immediate estimated cost with the estimated cost of eligible execution. All variants use the same execution conditions. NP tests the contribution of statistics probes; SH and GR examine the cost of restricting or removing lookahead, respectively.

Metrics. End-to-end latency measures elapsed time from the natural-language request to a result or failure, including LLM calls. Request latency starts at s_0 and is used only for XGAP and its variants. Planning time measures coordinator CPU time spent choosing actions. Backend calls count source requests, including metadata, probes, execution, and failed attempts. Clarification calls count requests to the simulated user. Call counts are averaged over supported requests, including failures.

Exact-match (EM) accuracy is the fraction of requests whose result matches the reference answer on the common supported set; failures and partial matches score zero. Overall completion rate is the fraction of all input requests that return a full result, regardless of correctness; unsupported requests and failures score zero. The action-budget scan reports completion rate on supported requests. Average interpretation loss is computed on the common supported request set. For each method, we average $d(Q^*, Q)$ over completed requests whose queries can be mapped to the common query representation. Appendix D.1 describes the query mapping and the treatment of queries that cannot be aligned [11].

Realized cost is $n_{\text{backend}} + n_{\text{clar}} + B/2^{20}$ work units, where the two counts denote backend and clarification calls, and B is the number of source-transferred bytes, including source-request targets and bodies and source-response bodies. We average this cost over the common supported request set, including failed and timed-out requests. These evaluation weights are fixed across methods and independent of the planner’s internal cost coefficients.

Protocol. Paired comparisons use requests supported by all compared methods, with the same intents and source snapshots. XGAP variants also share the same initial state s_0 . Execution order is randomized; cache and concurrency settings are fixed. Latency averages use requests completed by all compared methods. The action-budget experiment also includes failed and timed-out requests in its latency average and reports completion rates. We compute 95%

confidence intervals by paired bootstrap resampling of template families; Appendix D.1 details measurements and resampling [11].

Unless stated otherwise, parameter scans use D1+W3 with lookahead depth $D = 2$, action limit $H = 12$, candidate count $N = 8$, unresolved-choice count $u = 3$, and interpretation-loss tolerance $\epsilon = 1/3$. Each scan varies only one factor. SH retains $D = 1$; methods unaffected by the scanned parameter retain fixed configurations.

6.2 Efficiency

Overall Performance. Figures 6(a) and 6(b) compare end-to-end latency and backend calls across datasets. XGAP is faster than TS on all three datasets. This comparison includes LLM processing as well as planning and execution, so the improvement extends to the full request rather than only the coordinator. Within the XGAP variants, XGAP has the lowest latency on D1 and D3, whereas SH is faster on D2. Additional lookahead therefore pays off on two datasets, but the shallow variant gives a better latency trade-off on the third. XGAP also issues fewer backend calls than TS and GR on every dataset. Its advantage over NP and SH is small on D1 and D3 and reverses on D2. These results distinguish the external comparison from the ablation: the reduction relative to TS is consistent across datasets, but adding probes and deeper lookahead does not always minimize call count. A probe itself consumes a call; its benefit depends on the execution work avoided after updating statistics. The resource-cost comparison below accounts for both calls and transferred data.

Candidate and Binding Counts. We vary N and u separately to examine the cost of retaining query alternatives and resolving query choices. Request latency increases with N for all four variants (Figure 6(c)). From $N = 8$ to $N = 64$, XGAP’s latency increases by about 12%, compared with 22% for GR and 16% for SH. SH has lower latency throughout this scan, whereas XGAP grows more slowly. Thus the benefit of joint planning here is lower sensitivity to the candidate count, not the lowest absolute latency. GR’s low decision overhead does not give the best request latency: choosing inexpensive individual actions can incur more work later.

Latency also rises with u (Figure 6(d)). The increase is larger between one and three unresolved fields than at higher values, and SH again has the lowest latency. Unresolved choices require binding decisions in addition to physical planning. Probes refine execution statistics rather than resolve these choices. The observed growth therefore reflects the combined cost of handling ambiguity and executing the selected query. Across both scans, deeper lookahead is a resource-cost choice, not a universal latency win.

Planning Depth and Action Budget. Figure 6(e) shows increasing planning CPU time for XGAP and NP as D grows, until the configured 12 s planning limit is reached. SH stays at $D = 1$, and GR spends the least time choosing actions. Compared with their realized costs (Figure 7(f)), these results expose the main trade-off: reducing search saves coordinator computation, but can increase the calls and data exchange incurred afterward. The depth scan also shows why bounding lookahead matters; exploring more levels can consume the available planning time.

The action budget H limits performed actions rather than search depth. Figure 6(f) includes unsuccessful requests. XGAP’s completion rate rises from 0.22 at $H = 2$ to 0.94 at $H = 16$, while request

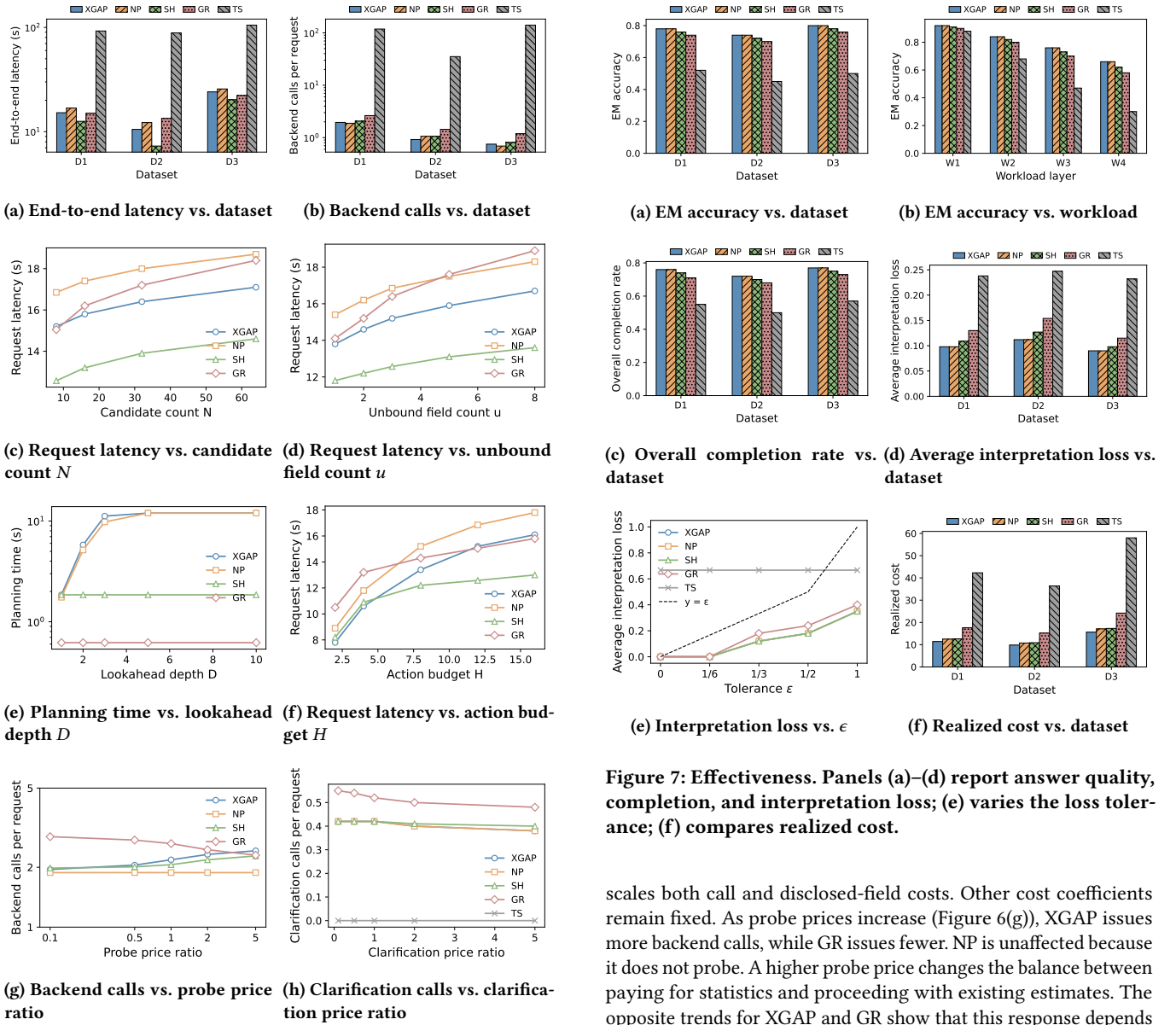


Figure 6: Efficiency results. Panels (a)–(b) compare all five methods across datasets. (c)–(h) vary one factor on D1+W3. Request latency and planning time compare XGAP variants. Panels (a),(b),(e),(g) use a logarithmic Y-axis.

latency increases. The short times at small H mainly reflect early termination, not efficient completion. A larger budget allows more requests to reach execution, but also permits more work before execution. This scan illustrates the cost of a tight action budget alongside the planning-computation trade-off controlled by D .

Acquisition Prices. Price ratios multiply the corresponding default action costs. A ratio of 1 denotes the default configuration. The probe-price scan scales probe costs; the clarification-price scan

Figure 7: Effectiveness. Panels (a)–(d) report answer quality, completion, and interpretation loss; (e) varies the loss tolerance; (f) compares realized cost.

scales both call and disclosed-field costs. Other cost coefficients remain fixed. As probe prices increase (Figure 6(g)), XGAP issues more backend calls, while GR issues fewer. NP is unaffected because it does not probe. A higher probe price changes the balance between paying for statistics and proceeding with existing estimates. The opposite trends for XGAP and GR show that this response depends on whether action selection includes continuation costs.

Clarification counts are much less sensitive to price (Figure 6(h)). The XGAP variants make 0.38–0.55 clarification calls per request, with nearly flat curves for XGAP, NP, and SH. TS makes no clarification calls in these runs. Thus price changes do not substantially reduce user interaction in this setting. Binding requirements still constrain execution even when clarification becomes expensive. The savings from choosing probes and physical plans coexist with a largely unchanged demand for binding information.

6.3 Effectiveness

Answer Quality and Completion. XGAP and NP have the same EM accuracy across the three datasets; SH and GR have lower accuracy (Figure 7(a)). Removing probes therefore preserves answer accuracy in these runs, although it increases realized cost. This

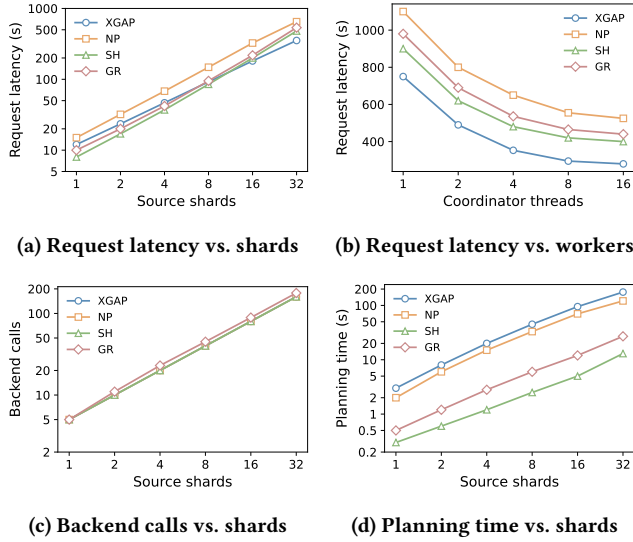


Figure 8: Scalability on a fixed financial graph. Panels (a), (c), and (d) use 4 coordinator workers; (b) uses 32 shards. Panels (a), (c), and (d) use logarithmic Y-axes.

separation matches the role of a probe: it informs physical-plan selection, while binding acquisition determines the query interpretation. Shorter lookahead and greedy action selection have a different effect: they reduce EM despite using the same execution conditions. Those conditions constrain admissible executions; they do not make every action sequence reach the same interpretation or complete successfully.

The workload comparison locates where XGAP’s accuracy advantage is largest. The XGAP–TS gap increases from 0.04 on W1 to 0.36 on W4 (Figure 7(b)). The systems are close on unambiguous, single-source requests and diverge on requests with multiple unresolved fields and cross-source access. Because W1–W4 vary both factors, the comparison supports XGAP on the combined setting rather than attributing the full gap to either ambiguity or federation alone. XGAP returns full results for 0.72–0.77 of all input requests, compared with 0.50–0.57 for TS (Figure 7(c)). Together with EM on the common supported set, this result shows both more completed requests over the input workload and higher answer accuracy where the methods are compared directly.

Interpretation Loss. XGAP and NP also have the same average interpretation loss (Figure 7(d)); SH, GR, and TS have higher loss. This agrees with their matching EM values: removing statistics probes changes execution cost without changing either quality measure in this comparison. Loss measures differences between query interpretations, whereas EM checks returned answers against the reference. Increasing ϵ permits larger deviations and increases the observed average loss (Figure 7(e)). The corresponding completion rates also increase (Appendix D) [11]. A larger tolerance therefore trades interpretation fidelity for completion; it is not an improvement in accuracy. At $\epsilon = 0$, XGAP completes fewer requests than TS, illustrating the cost of requiring zero loss. For the XGAP executions, the per-request audit records no violations of $d(Q^*, Q) \leq \rho_s(Q) \leq \epsilon$.

This check evaluates the bound for individual requests, separately from the averages in the plot.

Cost of Planning Choices. XGAP has the lowest realized cost in Figure 7(f). Removing probes (NP) or reducing lookahead to one step (SH) increases cost; choosing by immediate cost alone (GR) increases it further. The NP comparison identifies the contribution of statistics acquisition even when answer quality is unchanged. The SH comparison shows that considering only one action ahead leaves additional savings unused. GR retains probes, so its higher cost reflects the limitation of choosing actions by their own cost without evaluating what follows.

These savings must be considered alongside the earlier latency results. SH is faster on D2 and in the ambiguity scans, while XGAP uses fewer weighted resources in this dataset comparison. Realized cost excludes planning CPU time, which is reported separately. The results thus favor joint planning for reducing the combined call and exchange cost, with a workload-dependent latency trade-off. A separate fixed-plan-pool test in Appendix D examines sensitivity to cost-estimation error [11].

6.4 Scalability

For scalability, we generate a larger financial graph using FinBench, the benchmark used for D3. The graph contains 10M accounts, 100M transfers, and 210M attribute values, occupying 10.75 GB. We use one cross-source query template for both scans. We vary the shard count with four coordinator workers and vary the worker count with 32 shards. The graph and total hardware resources remain fixed. These scans remove the 12 s planning limit used in the depth experiment. Figure 8 reports request latency, backend calls, and planning CPU time.

Source Shards. Request latency increases with the shard count for all variants (Figure 8(a)). SH is fastest at 1–8 shards; XGAP becomes fastest at 16 and 32 shards. SH’s one-step lookahead reduces planning computation. As more shards are accessed, differences in physical-plan execution costs can grow. The crossover shows that planning overhead dominates at low shard counts, while execution savings dominate at high shard counts: XGAP pays more planning CPU (Figure 8(d)) but recovers it in execution.

Backend Calls. XGAP, NP, and SH each issue five calls per shard throughout the scan (Figure 8(c)). GR issues slightly more calls, reaching 178 at 32 shards. At 32 shards, XGAP, NP, and SH each issue 160 calls, yet XGAP’s request latency is 26.5% lower than SH’s and 45.7% lower than NP’s. Call counts alone therefore do not explain the latency differences; the gap comes from which physical plan each variant selects, not from how many requests it issues.

Planning Cost. Planning CPU time increases with the shard count for all variants (Figure 8(d)). XGAP’s planning time rises from 3.0 s at one shard to 176.0 s at 32 shards. At 32 shards, NP spends 121.0 s, SH 13.0 s, and GR 27.0 s on planning. XGAP uses 13.5 times as much planning CPU time as SH, yet its request latency is 26.5% lower. Minimizing planning computation does not simultaneously minimize request latency; deeper lookahead selects better physical plans under fragmentation.

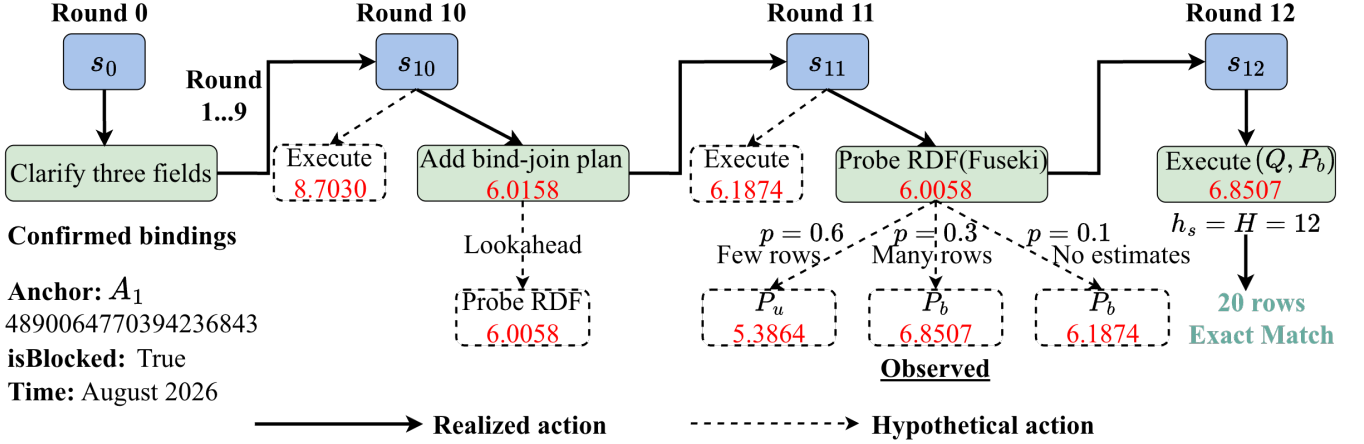


Figure 9: A recorded XGAP trace on a D3 W4 request with $D = 2$, $H = 12$, and $\epsilon = 1/3$. Costs are estimates in planning work units; the executed plan returns 20 rows that match the reference answer.

Coordinator Workers. At 32 shards, increasing coordinator workers reduces request latency for all variants (Figure 8(b)). XGAP falls from 750 s with one worker to 353 s with four workers, matching the 32-shard point in Figure 8(a). XGAP remains fastest throughout. Increasing coordinator workers from 8 to 16 reduces request latency by 4.8–5.4% across the four variants. The gains from coordinator parallelism diminish at higher worker counts.

6.5 Case Study

Figure 9 records a D3 W4 request for accounts reached by simple transfer paths of length two or three, with an RDF attribute filter and a time restriction. The query also requires an incoming transfer from an account other than the anchor. Round 0 clarifies the anchor, the target’s isBlocked value, and the time view, reducing eight candidates to one.

Rounds 1–9 construct physical alternatives(omitted). We show-case the final three rounds, as they show why plan construction and information acquisition remain coupled after the query interpretation has been fixed.

At round 10, immediate execution costs an estimated 8.7030 work units. Adding a bind-join plan costs 0.01 work units and enables a continuation with an estimated cost of 6.0058 work units. The total, 6.0158, is less than 8.7030, so XGAP adds the plan. The dashed probe is a lookahead branch used to evaluate this transformation; only the transformation is performed in this round.

At round 11, the new plan lowers the best immediate-execution estimate to 6.1874 work units. XGAP now evaluates the probe from the updated state. It estimates the cardinality of the RDF attribute fragment. The modeled outcomes are *Few rows*, *Many rows*, and *No estimate*, with probabilities 0.6, 0.3, and 0.1. Their execution estimates are 5.3864, 6.8507, and 6.1874 work units. Including the probe’s own cost, the expected cost is:

$$0.1 + 0.6 \cdot 5.3864 + 0.3 \cdot 6.8507 + 0.1 \cdot 6.1874 \approx 6.0058.$$

Since this is below 6.1874, XGAP performs the probe. The solid probe is the action considered hypothetically at round 10.

The outcomes favor different plans. P_u reads the RDF fragment without key restrictions, whereas P_b restricts the read to keys from the property-graph transfers. Few rows favor P_u ; many rows favor P_b . With no estimate, statistics remain unchanged and P_b retains its estimate of 6.1874 work units. The observed outcome is *Many rows*, so XGAP selects P_b with an updated estimate of 6.8507 work units. The probe was selected for its expected benefit, but the observed branch is more expensive than the prior estimate. Information changes the estimated costs; it need not lower the estimate in every outcome. At round 12, the action limit is reached. XGAP executes P_b and returns 20 rows matching the reference answer, with $\rho_s(Q) = 0$. The trace connects the two decisions: adding a physical alternative changes the value of probing, and the returned statistic determines which retained plan executes. XGAP performs one action at a time and replans, rather than committing to the entire lookahead branch.

7 CONCLUSION

We presented XGAP, a cost-based planner for graph queries with unresolved interpretations and incomplete execution information. Bounded lookahead jointly selects information-acquisition actions, physical-plan transformations, and when to execute under validation, interpretation-loss, and resource constraints. It performs one nonterminal action at a time and replans after observing its outcome. New information can therefore change both plan selection and subsequent acquisition decisions. Plan execution combines native queries and coordinator operators across property-graph and RDF sources. Experiments on real-world and synthetic graph workloads show lower end-to-end latency and higher exact-match accuracy than the two-stage baseline on requests supported by both systems. Ablations show that probing and multi-step lookahead reduce combined call and data-exchange costs. These savings require more planning computation; shallow lookahead is faster on some workloads. XGAP thus extends cost-based query optimization to deciding both what information to acquire and how to execute.

REFERENCES

- [1] Katrin Affolter, Kurt Stockinger, and Abraham Bernstein. 2019. A comparative survey of recent natural language interfaces for databases. *The VLDB Journal* 28, 5 (2019), 793–819.
- [2] Divy Agrawal, Sanjay Chawla, Bertty Contreras-Rojas, Ahmed Elmagarmid, Yasser Idris, Zoi Kaoudi, Sebastian Kruse, Ji Lucas, Essam Mansour, Mourad Ouzzani, et al. 2018. Rheem: enabling cross-platform data processing: may the big data be with you! *VLDB* 11, 11 (2018), 1414–1427.
- [3] Julien Aimonier-Davat, Brice Nédelec, Minh-Hoang Dang, Pascal Molli, and Hala Skaf-Molli. 2024. FedUP: Querying Large-Scale Federations of SPARQL Endpoints. In *Proceedings of the ACM Web Conference 2024*. Association for Computing Machinery, 2315–2324. <https://doi.org/10.1145/3589334.3645704>
- [4] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoč. 2017. Foundations of modern query languages for graph databases. *ACM Computing Surveys (CSUR)* 50, 5 (2017), 1–40.
- [5] Apache Software Foundation. [n.d.]. Apache Jena Fuseki. <https://jena.apache.org/documentation/fuseki2/> Accessed September 22, 2026.
- [6] Shivnath Babu, Pedro Bizarro, and David DeWitt. 2005. Proactive re-optimization. In *Proceedings of the 2005 ACM SIGMOD international conference on Management of data*. 107–118.
- [7] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J Mior, and Daniel Lemire. 2018. Apache calcite: A foundational framework for optimized query processing over heterogeneous data sources. In *Proceedings of the 2018 International Conference on Management of Data*. 221–230.
- [8] Dimitri P Bertsekas. 2013. Rollout algorithms for discrete optimization: A survey. In *Handbook of combinatorial optimization*. Vol. 5. Springer New York, 2989–3013.
- [9] Blai Bonet and Hector Geffner. 2012. Action Selection for MDPs: Anytime AO* Versus UCT. In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*. 1749–1755. <https://doi.org/10.1609/aaai.v26i1.8369>
- [10] Felix Brei, Lorenz Bühlmann, Johannes Frey, Daniel Gerber, Lars-Peter Meyer, Claus Stadler, and Kirill Bulter. 2025. ARUQULA – An LLM based Text2SPARQL Approach using ReAct and Knowledge Graph Exploration Utilities. In *Proceedings of the First International TEXT2SPARQL Challenge (CEUR Workshop Proceedings)*, Vol. 4094. CEUR-WS.org, 49–63. <https://ceur-ws.org/Vol-4094/paper4.pdf>
- [11] Haolai Che, Quanqing Xu, Hanchao Ma, and Yinghui Wu. 2026. Graph Query Planning Before the Query is Fully Known. https://github.com/anthonyche/XGAP/XGAP_FULL.pdf
- [12] Mohnish Dubey, Debayan Banerjee, Abdelrahman Abdelkawi, and Jens Lehmann. 2019. LC-QuAD 2.0: A Large Dataset for Complex Question Answering over Wikidata and DBpedia. In *ISWC 2019 (Lecture Notes in Computer Science)*, Vol. 11779. 69–78.
- [13] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC social network benchmark: Interactive workload. In *SIGMOD*.
- [14] Wenfei Fan, Jianzhong Li, Shuai Ma, Nan Tang, Yinghui Wu, and Yunpeng Wu. 2010. Graph pattern matching: From intractable to polynomial time. (*PVLDB*) 3, 1 (2010), 264–275.
- [15] Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoč. 2023. A Researcher’s Digest of GQL. In *ICDT*.
- [16] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindacker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An evolving query language for property graphs. In *Proceedings of the 2018 international conference on management of data*. 1433–1445.
- [17] Vijay Gadeppally, Peinan Chen, Jennie Duggan, Aaron Elmore, Brandon Haynes, Jeremy Kepner, Samuel Madden, Tim Mattson, and Michael Stonebraker. 2016. The BigDAWG polystore system and architecture. In *2016 IEEE High Performance Extreme Computing Conference (HPEC)*. 1–6.
- [18] Goetz Graefe. 1993. Query evaluation techniques for large databases. *ACM Computing Surveys (CSUR)* 25, 2 (1993), 73–169.
- [19] Goetz Graefe et al. 1995. The cascades framework for query optimization. *IEEE Data Eng. Bull.* 18, 3 (1995), 19–29.
- [20] F Maxwell Harper and Joseph A Konstan. 2015. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)* 5, 4 (2015), 1–19.
- [21] Fatma HÖzcan, Abdul Quamar, Jaydeep Sen, Chuan Lei, and Vasilis Efthymiou. 2020. State of the art and open challenges in natural language interfaces to data. In *SIGMOD*.
- [22] Zoi Kaoudi, Jorge-Arnulfo Quiané-Ruiz, Bertty Contreras-Rojas, Rodrigo Pardo-Meza, Anis Troudi, and Sanjay Chawla. 2020. ML-based cross-platform query optimization. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 1489–1500.
- [23] Hai Lan, Zhifeng Bao, and Yuwei Peng. 2021. A survey on advancing the dbms query optimizer: Cardinality estimation, cost model, and plan enumeration. *Data Science and Engineering* 6, 1 (2021), 86–101.
- [24] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Thomas Neumann, and Alfons Kemper. 2015. How good are query optimizers, really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215.
- [25] Fei Li and Hosagrahar V Jagadish. 2014. Constructing an interactive natural language interface for relational databases. *PVLDB* 8, 1 (2014), 73–84.
- [26] Shuxin Li, Gong Cheng, and Chengkai Li. 2020. Relaxing relationship queries on graph data. *Journal of Web Semantics* 61 (2020), 100557.
- [27] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making learned query optimization practical. In *SIGMOD*.
- [28] Neo4j, Inc. [n.d.]. Neo4j Documentation. <https://neo4j.com/docs/> Accessed September 22, 2026.
- [29] Shipeng Qi, Bing Tong, Jiatao Hu, Heng Lin, Yue Pang, Wei Yuan, Songlin Lyu, Zhihui Guo, Ke Huang, Xujin Ba, Qiang Yin, Youren Shen, Yan Zhou, Tao Lv, Jia Li, Lei Zou, Yongwei Wu, Gábor Szárnyas, Xiaowei Zhu, Wenguang Chen, and Chuntao Hong. 2025. The LDBC Financial Benchmark: Transaction Workload. *VLDB* 18, 9 (2025), 3007–3020.
- [30] Abdul Quamar, Vasilis Efthymiou, Chuan Lei, and Chuan Lei. 2022. Natural language interfaces to data. *Foundations and Trends in Databases* 11, 4 (2022), 319–414.
- [31] Irina Saparina and Mirella Lapata. 2025. Disambiguate first, parse later: Generating interpretations for ambiguity resolution in semantic parsing. In *ACL*.
- [32] Andreas Schwarte, Peter Haase, Katja Hose, Ralf Schenkel, and Michael Schmidt. 2011. FedX: Optimization Techniques for Federated Query Processing on Linked Data. In *The Semantic Web – ISWC 2011 (Lecture Notes in Computer Science)*, Vol. 7031. Springer, 601–616. https://doi.org/10.1007/978-3-642-25073-6_38
- [33] P Griffiths Selinger, Morton M Astrahan, Donald D Chamberlin, Raymond A Lorie, and Thomas G Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 23–34.
- [34] Yuchen Tian, Kaixin Li, Hao Chen, Ziyang Luo, Hongzhan Lin, Sebastian Schelter, Lun Du, and Jing Ma. 2025. AmbiGraph-Eval: Can LLMs Effectively Handle Ambiguous Graph Queries? *arXiv preprint* (2025).
- [35] Yinghui Wu, Shengqi Yang, and Xifeng Yan. 2013. Ontology-based subgraph querying. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE, 697–708.
- [36] Ziyu Yao, Yu Su, Huan Sun, and Wen-tau Yih. 2019. Model-based interactive semantic parsing: A unified framework and a text-to-SQL case study. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 5447–5458.

A ANALYSIS OF UNIFIED PLANNING

Proof of Theorem 1. Reduce from weighted set cover with universe X , sets $S_1, \dots, S_m$, nonnegative weights $w_1, \dots, w_m$, and threshold W . Create one complete candidate query whose required fields correspond to X , and one initially retained zero-cost physical plan on one backend. Every field has a unique candidate value but requires an acquired validation record. Set $\Lambda = \emptyset$ and $\epsilon = 0$. The authorized rule list contains no rule deriving these records from candidate uniqueness or the action model.

For each S_i , register one binding-acquisition action of cost w_i that returns authorized attestations validating exactly its fields. Each action is usable once. No other action can obtain these attestations. Set $H = m$, make remaining budgets nonbinding, and set all other costs to zero. This catalog contains the given m requests, not all subsets of fields.

If a cover has weight at most W , perform its actions and execute the plan. Every required record is then present, and the trace has cost at most W . Conversely, deterministic outcomes give one trace for any successful policy. Every required field must be validated by an action on that trace. The corresponding sets therefore cover X , and their total weight equals the trace cost. The construction is polynomial and proves the claim. This hardness concerns unrestricted policy optimization, not evaluation of the fixed-depth heuristic.

Proof of Theorem 2. Initially $Q^* \in Q_0$. A correct authorized observation or sound validation inference cannot exclude Q^* . Metadata, probes, and physical transformations do not remove interpretations. In particular, discarding a physical plan does not discard its query from Q_s . Induction on successful transitions preserves $Q^* \in Q_s$, Γ , and all previously validated bindings. A failed call ends the request without supplying a state transition.

Before each nonterminal action, admission is checked against the remaining budgets. Admission bounds cover every permitted outcome. For a cumulative resource, the sum of consumption so far and the action’s valid upper bound must not exceed its budget. For a peak resource, the maximum of the recorded peak and the action upper bound must not exceed its budget. Calls enforce these limits, including when they return failure. Thus induction also preserves the resource constraints along each executed prefix.

The final pair (Q, P) is selected from $\mathcal{F}_\epsilon(s)$. Membership in this set establishes the required validations and a sound compilation or transformation derivation for $P \equiv Q$. On successful execution over the specified snapshots, the result is therefore $\text{Eval}(Q, \mathcal{G})$. It also establishes

$$d(Q^*, Q) \leq \max_{Q' \in Q_s} d(Q', Q) = \rho_s(Q) \leq \epsilon.$$

The terminal resource bounds fit the budgets left by the prefix. The reasoning depends on evidence and eligibility, not on the accuracy of cost estimates or the leaf heuristic.

Local Operation Counts. Let $N = |Q_0|$, $u = |U(Q^\circ)|$, and let v count the explicitly listed information requests in addition to the per-choice requests. There are at most NK retained plans of size at most ℓ . The fixed transformation rules inspect operator positions and adjacent join pairs. Even scanning all pairs of positions uses at most $O(NK\ell^2)$ matches before placement alternatives are considered. Each placement change selects one entry from the explicit list of equivalent locations. The number W_{gen} of generated variants, including rejected variants, is therefore polynomial in the input size.

Information-acquisition actions and plan-transformation actions give $A = O(u + v + W_{\text{gen}})$. The number Δ is bounded by the explicit modeled outcome lists. A request for a single choice can enumerate its values in the candidate set; additional attestations or unavailable responses must be included in its outcome description. A deterministic plan transformation has one outcome.

Filtering candidates scans at most N complete assignments. Checking ρ_s for every candidate requires at most N^2 evaluations of d ; each evaluation has the polynomial cost stipulated in Section 4. Each generated plan requires rule, size, resource, and cost checks on polynomial-size records. Deduplication can sort plan encodings, and retention sorts the estimates, using $O(W_{\text{gen}} \log(2 + W_{\text{gen}}))$ comparisons per batch, with polynomial work per comparison. The supplied validation procedure has the same polynomial-time requirement. For a bounded list of ground rules and possible facts, repeated scans suffice: each productive pass adds a fact, so the number of productive passes is bounded by the fact list.

Let S_{state} bound the encoded state size. Besides the explicit input and at most NK retained plans, a reached state contains at most H action records and the resulting validation and statistics records. Their sizes and numeric bit lengths have the polynomial bounds

specified in the main text. Response checks include verifying the permitted range or category of a probe estimate. Each online round processes one returned response. Copying states, updating dependencies, and checking versions also take polynomial time. Together these operations give a polynomial bound T_{local} .

Planning Time and Space. Each lookahead state occurrence has at most $A\Delta$ successor occurrences, so a depth- D lookahead contains at most $B_D = \sum_{j=0}^D (A\Delta)^j$ state occurrences. Generation and local checks are charged to the current state; each transition and its cost backup are charged to the successor. A round thus costs $O(B_D T_{\text{local}})$. After at most H successful nonterminal transitions, the recurrence permits only execution or failure. Including the final check gives Equation 3. Fixed D and the stated polynomial bounds make the total polynomial. In particular, the value of H , rather than only its binary encoding length, must satisfy its stated polynomial bound.

A stored state includes the retained plans and observation records. The tree contains $O(AB_D)$ action and outcome descriptors; their individual size is bounded by S_{state} . Hence $O((A + 1)B_D S_{\text{state}})$ space suffices for a materialized lookahead and its reached-state evidence. Discarding a round’s hypothetical states before the next round avoids storing a full conditional policy over horizon H .

This counts coordinator planning operations. Calls either return or fail within their deadlines, and response limits bound parsing work. Backend execution and result materialization are outside this planning-time bound.

Proof of Lemma 3. Let $\hat{P}$ minimize estimated cost and P_{opt} minimize true cost among the eligible retained plans for Q at s . Both plans satisfy the uniform error assumption. Therefore

$$\begin{aligned} c_{\text{term}}(s, Q, \hat{P}) &\leq \hat{c}_{\text{term}}(s, Q, \hat{P}) + \eta \\ &\leq \hat{c}_{\text{term}}(s, Q, P_{\text{opt}}) + \eta \\ &\leq c_{\text{term}}(s, Q, P_{\text{opt}}) + 2\eta. \end{aligned}$$

The middle step uses the minimizing property of $\hat{P}$. The comparison holds within this fixed eligible set.

B SYSTEM DETAILS

B.1 Candidate Construction and Interpretation Loss

Each unresolved choice occupies a declared field in Q° and has a finite domain of type-compatible values. A field may select an entity, predicate, relation constraint, or logical source. Candidate construction substitutes these values, checks Γ , and produces complete logical queries. Fixed query components remain unchanged. Before expansion, it checks the product of domain sizes against the configured family limit. An explicitly supplied candidate list can also be used. Canonical field vectors and query encodings remove duplicates.

Coverage confirmation identifies the request and the admitted family and is stored in $\mathcal{E}_{s_0}$. Binding observations identify the requested fields, returned values, and authorized source. Validation rules operate on explicit fields and evidence records. A derived validation records its premises and rule. For the polynomial construction used in the analysis, the sets of ground rules and possible

facts have polynomial size; forward chaining adds each fact at most once.

For the field-based loss in Section 5, computing all pairwise losses takes $O(N^2u)$ field comparisons and additions. The computation of ρ_s uses all candidates in Q_s , as specified in Section 3.

B.2 Information Requests and State Updates

A binding action lists the requested fields and the distinct response tuples represented in the candidate family. Requests for validation evidence specify their permitted responses separately. A capability request names the source, operator signature, and required semantics; its response is supported, unsupported, or unknown.

A probe names its source fragment or predicate and specifies cardinality or selectivity intervals. These intervals cover the permitted numeric responses. The action also includes an unavailable-information outcome when its interface permits one. Probe records distinguish exact counts, intervals from capped counts, and estimates. They retain the returned value or interval alongside the estimate used for planning. Appendix C defines category representatives and statistics updates.

Observations record the source snapshot and, for probes, the estimation method. Plans record dependencies on source capabilities, mappings, and statistics. A capability update revisits the affected native fragments. A statistics update invalidates dependent cardinalities and costs. Cached entries include the query, source and mapping identities, and the versions of their dependencies.

B.3 Compilation and Physical Alternatives

Native fragments expose named columns with declared types. Their result decoders produce the binding rows consumed by coordinator operators. Validated mappings associate source identifiers with canonical join keys; parameter encoders translate keys into identifiers accepted by the target source. Type conversion and missing-value handling follow the query’s semantics. Binding rows preserve the multiplicities required by the query. Match identifiers distinguish parallel edges during path and aggregate computation.

A source predicate used only as a necessary filter retains the corresponding coordinator predicate. Sharing a native read requires identical fragment semantics, parameters, and source snapshots.

The compiler records the parent plan, applied rule, and satisfied conditions for each variant. Canonical plan encodings support deduplication.

B.4 Bind-Join Preservation

Let R be the driver relation, S the target, and k their canonical join key. Under an inner equijoin with nonmatching null keys,

$$R \bowtie_k S = R \bowtie_k (S \bowtie_k \pi_k(R)).$$

Every matching pair on the left has a target key present in R and survives the semijoin. The semijoin removes only target rows without a matching driver key and preserves the multiplicity of every retained row. Thus both sides produce the same pairs with the same multiplicities. Deduplicating lookup keys changes only the membership test; the final join still uses the original driver. Subsequent aggregates therefore receive the same contributions.

Table 3: Internal planner weights. Each coefficient converts the listed quantity to planning work units and is fixed for a planning run.

Weight	Charged quantity	Unit
$\omega_{\text{call},b}$	Calls to backend b	work/call
$\omega_{\text{src},b}$	Source record operations	work/operation
ω_{net}	Exchanged bytes	work/MiB
ω_{cpu}	Coordinator and compiler CPU	work/ms
ω_{user}	Clarification calls	work/call
ω_{field}	Disclosed fields	work/field

The reduction applies to a target match behind key-preserving projections and filters. The target has no other consumer or answer root, so the restriction cannot remove rows needed elsewhere. Cross-source keys use validated identifier mappings. Batching partitions the distinct canonical keys into disjoint groups and combines the target responses before the original join. The key bound and batch size determine the bound on lookup calls; any permitted retries are included in that bound.

B.5 Execution Records and Resource Accounting

Initialization records candidate construction, coverage confirmation, initial backend information, and initial plans before the online ledger begins at s_0 . Each performed action records its outcome and resource use. The selected query and plan identify the computation that produced the materialized result.

Resource bounds used for admission cover permitted responses and include batched calls. Sample estimates used for costing do not supply these bounds. The runtime enforces call deadlines, response-size limits, and source snapshot requirements. Information acquisition, performed transformations, and execution contribute to $C(\tau)$; initialization and hypothetical lookahead computation are recorded separately, following Section 3.

C COST MODEL

Objective and Units. The planner estimates the weighted resource cost of actions and execution in planning work units. For a native fragment f , let $\hat{m}_f$ be its number of calls and $\hat{x}_f$ its source processing work, measured in record operations. Let $\hat{B}_P$ be the bytes exchanged across source boundaries. Let $\hat{t}_{\text{coord}}$ and $\hat{t}_{\text{comp}}$ be the remaining coordinator processing and compilation CPU time in milliseconds. Then

$$\begin{aligned} \hat{c}_{\text{term}}(s, Q, P) = & \sum_{f \in P} (\omega_{\text{call},b(f)} \hat{m}_f + \omega_{\text{src},b(f)} \hat{x}_f) \\ & + \omega_{\text{net}} \frac{\hat{B}_P}{2^{20}} + \omega_{\text{cpu}} (\hat{t}_{\text{coord}} + \hat{t}_{\text{comp}}). \end{aligned}$$

The sum ranges over native-fragment nodes in the physical DAG, each counted once; $b(f)$ is the backend of f . Table 3 specifies the nonnegative weights and their units. Source processing, network transfer, and coordinator CPU are charged separately. Their sum estimates resource cost, not elapsed time under concurrent execution.

Cardinalities and Row Widths. A topological pass over the physical DAG estimates each operator’s output cardinality, serialized row width, and distinct join-key counts. Source statistics are scoped to the source snapshot, labels or predicates, and filter conditions. A selection with selectivity $\hat{\sigma}$ produces $\hat{n}\hat{\sigma}$ rows from an input of size $\hat{n}$. Bag projection preserves the row count; duplicate elimination uses the estimated number of distinct projected tuples.

An inner join produces $\hat{n}_L\hat{n}_R\hat{\sigma}_{\text{join}}$ rows. For non-null keys with uniform frequencies and key-domain containment, the approximation sets $\hat{\sigma}_{\text{join}} = 1/\max\{1, \hat{v}_L, \hat{v}_R\}$, where $\hat{v}_L$ and $\hat{v}_R$ are distinct-key counts. Key-frequency statistics refine this approximation when available. Declared uniqueness constrains the estimate: a unique right key limits the output to the left input size. Grouping uses the distinct count of the group-key tuple, capped by the input size; an aggregate without group keys produces one row.

For bounded paths, the pass estimates each successive expansion from the preceding matches and the fanout of the relevant edge label and direction. Degree statistics distinguish source and target endpoints. The estimate includes intermediate matches even when a later filter or output limit removes them. Row widths include projected values, join keys, and their source-specific serialization. A projection can therefore reduce transfer cost without changing cardinality.

Operator Work and Source Calls. Let W_v be the estimated operation count for operator v . A scan counts records examined, including records rejected by filters. An indexed lookup counts lookup keys and records reached through the index. A filter or projection counts input rows. Hash-join work is the sum of build, probe, and output rows. Nested-loop work is the input-cardinality product plus output rows. Hash aggregation counts input rows and output groups. A full sort uses $\hat{n}\log_2 \max\{2, \hat{n}\}$ operations; a bounded top- k heap uses $\hat{n}\log_2 \max\{2, k\}$.

For path traversal, W_v sums the estimated adjacency records examined over all expansion steps. Visits from different partial paths count separately, including extensions rejected by path constraints or filters.

The fragment descriptor specifies the operator implementations and access paths used for estimation. An unknown access path uses the scan model. For a fragment f , $\hat{x}_f$ sums these operator counts over all $\hat{m}_f$ calls, using each call’s estimated input and output sizes. Thus batched requests include the work of every batch.

For each source call, the byte estimate includes the encoded request and the returned rows. Returned bytes are estimated as row count times serialized row width. A bind request also includes its encoded keys. Summing these request and response sizes gives $\hat{B}_p$. A shared native-fragment node reuses its results across consumers; each consumer incurs its own coordinator processing cost. Network transfer is charged at the source boundary, not again at the corresponding exchange operator.

Hash Joins and Bind Joins. A hash join retrieves both inputs and joins them at the coordinator. A bind join retrieves its driver R , extracts distinct mapped keys, and requests matching rows from the target S . If the estimated number of keys is $\hat{v}_R$ and a request holds at most q keys, the estimated number of target calls is $\lceil \hat{v}_R/q \rceil$.

The batch capacity q respects both the key-count and request-size limits. An empty driver requires no target call.

With uniform target-key frequencies and key-domain containment, the restricted target has estimated size

$$\hat{n}_S \min \left\{ 1, \frac{\hat{v}_R}{\max(1, \hat{v}_S)} \right\}.$$

Key-overlap and frequency statistics replace this approximation when available. The final join still uses the original driver cardinality, including duplicates. Restricting target output reduces source work only when the access path supports the restriction. If each batch scans the target, source work includes one scan per batch. The estimate thus accounts for batching, key transfer, target work, and the final join.

Coordinator Work and Compilation. For a coordinator operator v , the CPU estimate is $\alpha_{\text{op}(v)} + \beta_{\text{op}(v)}W_v$, with nonnegative coefficients in ms and ms/operation, respectively. Summing over coordinator operators gives $\hat{t}_{\text{coord}}$.

Compilation estimates use a nonnegative linear model of the number of operators and the bytes of native-query text still to be generated. A compiled artifact in the current state contributes no remaining compilation cost. A performed transformation pays for constructing its variant and generating any new native fragments.

Action Costs. A clarification that discloses k fields has estimated cost $\omega_{\text{user}} + k\omega_{\text{field}}$, plus any coordinator processing required for its response. A metadata request or probe uses the same call, source-work, byte, and CPU terms as execution. In particular, a scalar probe response can require a large source scan. A plan transformation has cost $\omega_{\text{cpu}}\hat{t}_{\text{transform}}$, where the time estimate covers variant construction and new compilation. These calculations supply $\hat{c}(s, a, o)$; response-dependent quantities use the specified outcome o . Initialization, completed actions, and hypothetical lookahead computation are excluded from remaining costs.

Probe Updates. A probe replaces the statistic for its target. The estimator then recomputes dependent cardinalities, key counts, byte volumes, and operator costs in topological order. Hypothetical outcomes use category representatives; performed probes use the estimates returned by their adapters. An unavailable estimate leaves the previous statistics intact.

A capped count needs separate treatment. If the cap is $t + 1$, a returned value of $t + 1$ establishes only that the count exceeds t . The adapter records this interval and uses a configured estimate within it. An exact count, including an unsaturated capped count, replaces the estimate directly.

Configuration and Calibration. The model configuration fixes resource weights, operator coefficients, default statistics, row-width estimates, and bind-batch limits. Resource weights express the planning objective. CPU coefficients describe implementation costs: calibration measures the corresponding operators on development inputs and fits nonnegative coefficients to their operation counts. Compilation calibration varies the number of operators and generated query length. These measurements exclude remote wait time. A configuration is frozen before evaluating test requests.

Each numeric outcome category has a configured representative within its interval. Outcome probabilities, when supplied, assign

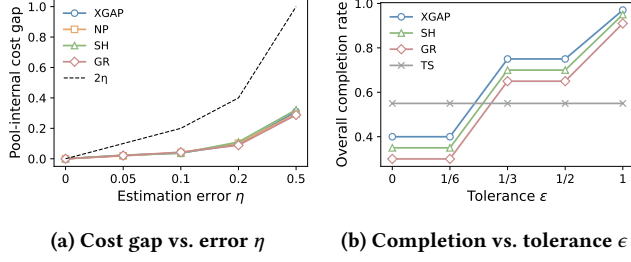


Figure 10: Complementary experiments. Panel (a) perturbs normalized measured costs within a fixed plan pool. Panel (b) reports completion under different loss tolerances.

weights to these categories. Arithmetic uses fixed precision and finite caps to bound the numeric representation size. The probability evaluator and cost calculations operate on the bounded records assumed in Section 4.

D COMPLEMENTARY EXPERIMENTS

D.1 Evaluation Protocol

Interpretation-Loss Evaluation. After execution, we parse TS’s generated SPARQL and normalize variable names and source identifiers. We align the query with the workload’s fixed query structure and extract its entity, predicate, relation-constraint, and logical-source choices. We then evaluate the same field-based loss used for XGAP. Queries that cannot be aligned are excluded from the loss average and counted separately. The loss average uses the number of aligned completed requests as its denominator.

Repeated Measurements and Confidence Intervals. Each request-method-configuration combination is run three times. We first average repeated measurements within each request. For experiments with multiple template families, we construct 95% percentile

confidence intervals from 2,000 bootstrap samples. We resample template families within each dataset and workload class. Each sampled family includes all of its requests. Paired comparisons use the same sampled families across methods.

D.2 Plan Selection under Estimation Error

Fix a complete query Q and a pool $\mathcal{P}$ of equivalent, execution-feasible plans. Let $c(P)$ be the median execution time of P over three runs and let $Z = \max_{P \in \mathcal{P}} c(P) > 0$. For each error level η , select $\hat{P}$ by minimizing $\max\{0, c(P)/Z + \eta z_P\}$, where each z_P is sampled uniformly from $[-1, 1]$ once and reused across error levels. The pool-internal cost gap is the normalized execution-time difference

$$\frac{c(\hat{P}) - \min_{P \in \mathcal{P}} c(P)}{Z}.$$

The perturbed costs differ from the normalized measured costs by at most η . Lemma 3 therefore bounds this gap by 2η .

Figure 10(a) shows increasing gaps that remain below 2η . The variants use the same fixed plan pool for this test. This experiment measures plan-selection sensitivity to controlled cost perturbations; it does not measure the prediction error of the analytical cost model.

D.3 Completion under Different Loss Tolerances

Figure 10(b) reports overall completion rate under different loss tolerances; TS does not use ϵ and is shown as a fixed reference. At $\epsilon = 0$, XGAP completes 0.40 of requests, below TS at 0.55, because strict validation rejects requests whose designated bindings cannot be confirmed. Completion rises in steps as tolerance loosens: 0.75 at $\epsilon = 1/3$, where XGAP overtakes TS, and 0.97 at $\epsilon = 1$. NP coincides with XGAP, consistent with probes affecting only the physical plan; SH and GR follow the same shape but stay below at every tolerance. Together with Figure 7(e), this bounds the default $\epsilon = 1/3$ from both sides: tighter tolerance costs coverage, looser tolerance costs interpretation loss.